

AAEC4484/AAEC(STAT)5484: Applied Economic Forecasting

Alex Fuentes

Homework #5 - Spring 2026

Instructions: Where necessary ensure that your graphs and visuals have proper titles and axis labels. Refer to the output, whenever appropriate, when discussing the results. **Creativity (coupled with relevance) will be rewarded.**

Q1: Classical Multiplicative Decomposition

In this exercise, we will explore the classical decomposition.

1. Return to the New Private Housing Units Authorized by Building Permits for Virginia (VABPPRIV) series. **In a single step**, import the data from the FRED database using the `quantmod` package. Convert to a `tsibble` object, with the index converted to a `yearmonth` object, and store it as `permit`.

```
permit <- quantmod::getSymbols("VABPPRIV", src = "FRED", auto.assign = FALSE) %>%  
  as_tibble(rownames = "Month") %>%  
  mutate(Month = tsibble::yearmonth(Month)) %>%  
  as_tsibble(index = Month)
```

If you are still encountering issues with the `quantmod` package, you can use the direct website link for the csv data, as I showed in class.*

2. Produce an autoplot of the `permit` series along with a `gg_subseries` plot. Use the `patchwork` package to combine the two plots. **Remember your titles and axis labels.**
 - Briefly comment on any pattern you observe.
 - Xiaoyi suggested that a multiplicative decomposition is appropriate for this series? Do you agree with her assessment? **Remember to explain your reasoning. A simple yes or no will not suffice.**

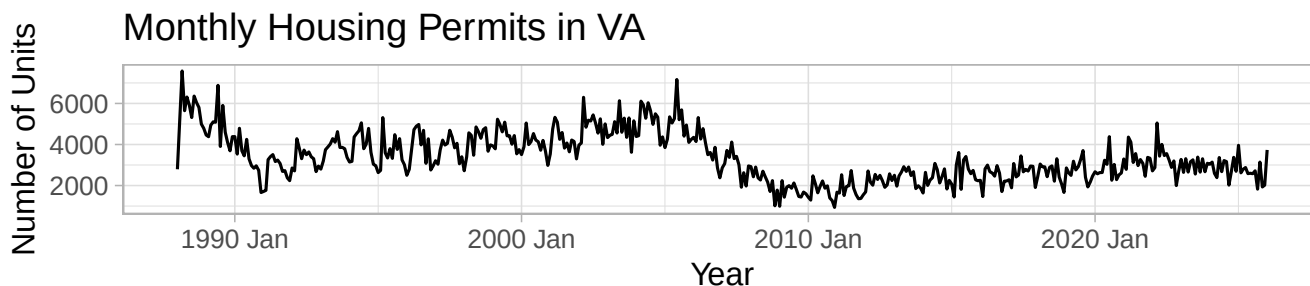
The ‘`permit`’ series exhibits seasonal spikes with a peak in the middle of summer and low going into January. There are trends as well, with a notable decrease around the year 2008 possibly due to the global financial crisis with a slight recovery after as well.

Yes, I do agree with Xiaoyi, a multiplicative decomposition should be used. The seasonal variance seems to be much higher before the decrease and it seemingly seems to get more strict after the crisis. I believe a multiplicative decomposition would explain the underlying dynamics better due to how it holds seasonal effects constant.

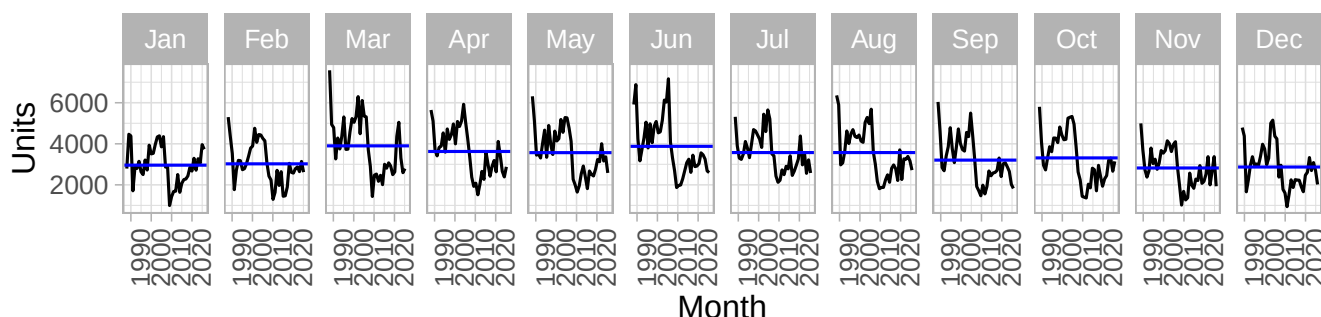
```
p1 <- permit %>%  
  autoplot(VABPPRIV) +  
  labs(title = "Monthly Housing Permits in VA",  
       y = "Number of Units", x = "Year")  
  
p2 <- permit %>%
```

```
gg_subseries(VABPPRIV) +
labs(title = "Seasonal Subseries Plot",
y = "Units", x = "Month")
```

p1 / p2



Seasonal Subseries Plot



3. With visuals and our observations in tow, we are ready to perform our decomposition.

Step i. Using an appropriate centered moving average, $m - MA$ model, extract the trend-cycle component, T_t . Recall that we might need to doubly smooth the series to get it to be symmetric.

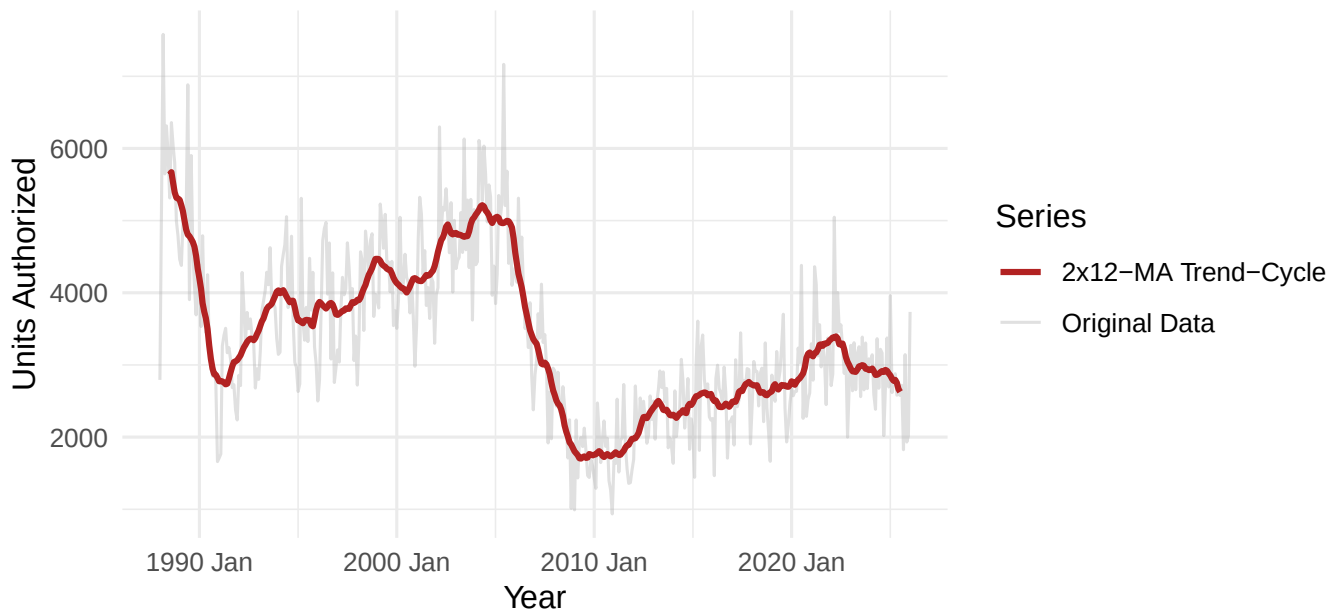
```
permit_trend <- permit %>%
  mutate(
    Trend_Cycle = slider::pslide_dbl(
      list(VABPPRIV),
      .f = ~mean(.x),
      .before = 5, .after = 6, .complete = TRUE
    ) %>%
    slider::slide_dbl(mean, .before = 1, .after = 0, .complete = TRUE))
```

Present a plot of your trend-cycle element on top of the original data.

```
permit_trend %>%
  ggplot(aes(x = Month)) +
  geom_line(aes(y = VABPPRIV, color = "Original Data"), alpha = 0.4) +
  geom_line(aes(y = Trend_Cycle, color = "2x12-MA Trend-Cycle"), size = 1) +
  scale_color_manual(values = c("Original Data" = "gray70", "2x12-MA Trend-Cycle" = "firebrick")) +
  labs(
    title = "Virginia Housing Permits: Trend-Cycle Extraction",
    subtitle = "Extracted using a Symmetric 2x12 Moving Average",
    y = "Units Authorized",
    x = "Year",
    color = "Series") +
  theme_minimal()
```

Virginia Housing Permits: Trend–Cycle Extraction

Extracted using a Symmetric 2x12 Moving Average

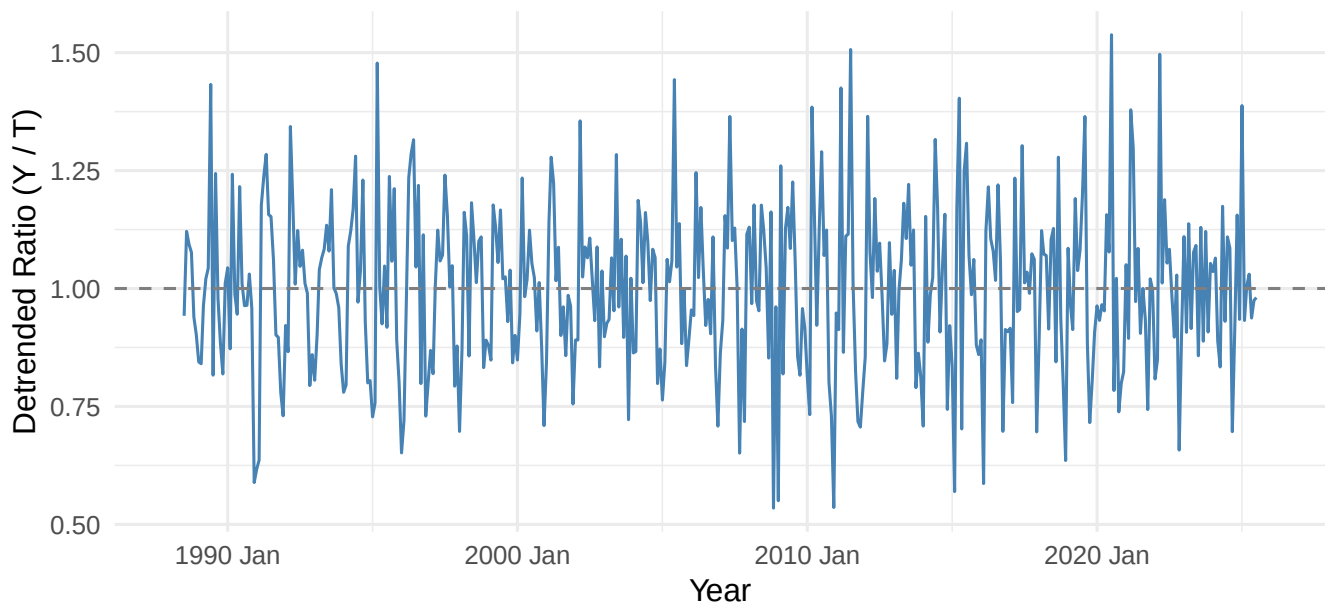


Step ii. Using the trend-cycle computed above, compute and plot the **detrended** series

```
permit_detrended <- permit_trend %>%  
  mutate(Detrended = VABPPRIV / Trend_Cycle)  
  
permit_detrended %>%  
  ggplot(aes(x = Month, y = Detrended)) +  
  geom_line(color = "steelblue") +  
  geom_hline(yintercept = 1, linetype = "dashed", color = "gray50") +  
  labs(  
    title = "Detrended Virginia Housing Permits (Multiplicative)",  
    subtitle = "Values above 1 indicate observations higher than the trend-cycle",  
    y = "Detrended Ratio (Y / T)",  
    x = "Year"  
  ) +  
  theme_minimal()
```

Detrended Virginia Housing Permits (Multiplicative)

Values above 1 indicate observations higher than the trend-cycle



Step iii (a). Using a TSLM model, extract the seasonal component, S_t . Also, plot your seasonal component.

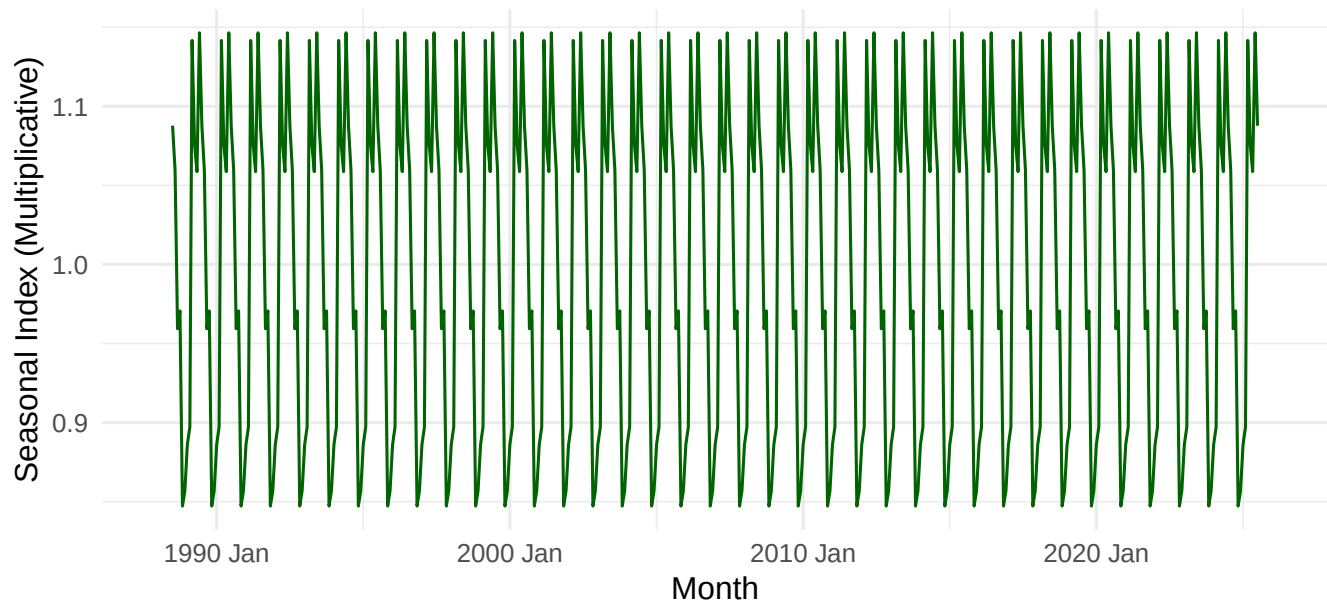
```
fit_seasonal <- permit_detrended %>%
model(tslm = TSLM(Detrended ~ season()))

permit_seasonal <- fit_seasonal %>%
augment() %>%
select(Month, .fitted) %>%
rename(Seasonal = .fitted)

permit_seasonal %>%
ggplot(aes(x = Month, y = Seasonal)) +
geom_line(color = "darkgreen") +
labs(
  title = "Estimated Seasonal Component ( $S_t$ )",
  subtitle = "Extracted using TSLM with seasonal dummy variables",
  y = "Seasonal Index (Multiplicative)",
  x = "Month") +
theme_minimal()
```

Estimated Seasonal Component (St)

Extracted using TSLM with seasonal dummy variables

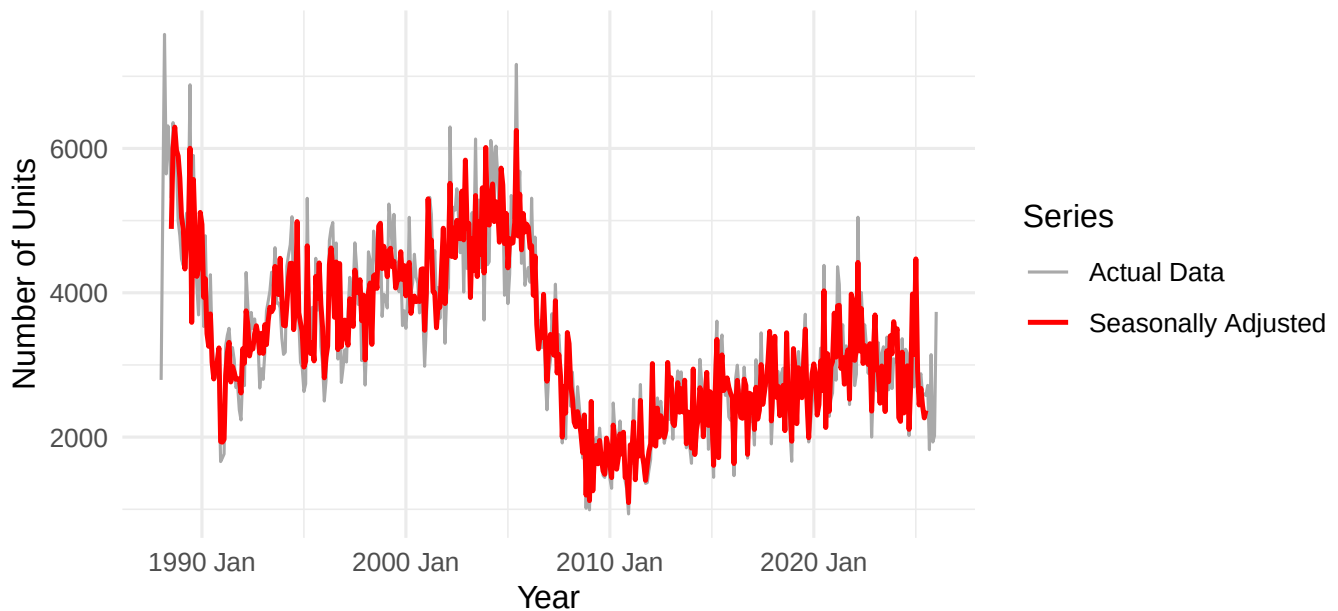


Step iii(b). Produce a plot of the actual data (in darkgray) and overlay the seasonally-adjusted variable.

```
permit_adj <- permit %>%  
  left_join(permit_seasonal, by = "Month") %>%  
  mutate(Seasonally_Adjusted = VABPPRIV / Seasonal)  
  
permit_adj %>%  
  ggplot(aes(x = Month)) +  
  geom_line(aes(y = VABPPRIV, color = "Actual Data"), size = 0.5) +  
  geom_line(aes(y = Seasonally_Adjusted, color = "Seasonally Adjusted"), size = 0.8) +  
  scale_color_manual(values = c("Actual Data" = "darkgray", "Seasonally Adjusted" = "red")) +  
  labs(  
    title = "Virginia Housing Permits: Seasonal Adjustment",  
    subtitle = "Multiplicative adjustment (Actual / Seasonal Index)",  
    y = "Number of Units",  
    x = "Year",  
    color = "Series") +  
  theme_minimal()
```

Virginia Housing Permits: Seasonal Adjustment

Multiplicative adjustment (Actual / Seasonal Index)

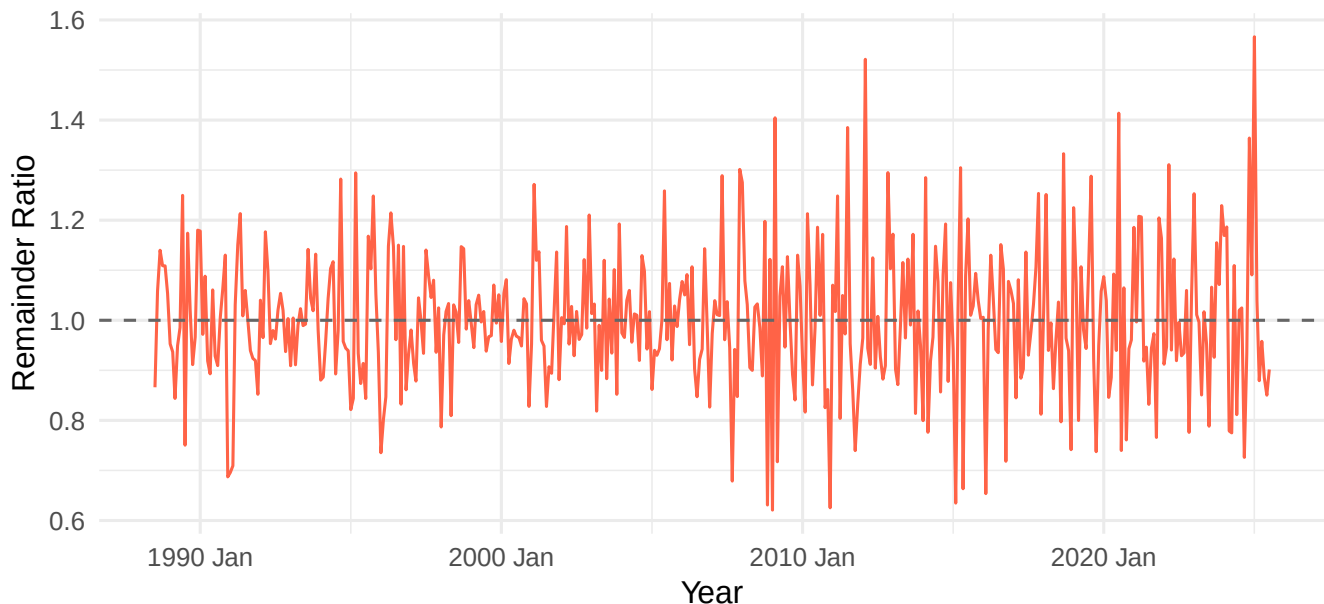


Step iv. Lastly, compute and plot the remainder component.

```
permit_final <- permit_adj %>%  
left_join(permit_trend %>% select(Month, Trend_Cycle), by = "Month") %>%  
mutate(Remainder = Seasonally_Adjusted / Trend_Cycle)  
  
permit_final %>%  
filter(!is.na(Remainder)) %>%  
ggplot(aes(x = Month, y = Remainder)) +  
geom_line(color = "tomato") +  
geom_hline(yintercept = 1, linetype = "dashed", color = "gray40") +  
labs(  
title = "Remainder Component (Rt): Virginia Housing Permits",  
subtitle = "Multiplicative Remainder: Values around 1.0 indicate random noise",  
y = "Remainder Ratio",  
x = "Year") +  
theme_minimal()
```

Remainder Component (Rt): Virginia Housing Permits

Multiplicative Remainder: Values around 1.0 indicate random noise



Step v. Produce a gridded plot of the data and the three components. Store your plot as `plot1`. Do not present the plot here.

Please order your variables (graphs) as **Data » Trend » Seasonal » Remainder**.

```
# Creating the individual components for the grid
p_data <- permit_final %>%
  ggplot(aes(x = Month, y = VABPPRIV)) +
  geom_line(color = "gray30") +
  labs(title = "Data (Yt)", y = "Units", x = "") +
  theme_minimal()

p_trend <- permit_final %>%
  ggplot(aes(x = Month, y = Trend_Cycle)) +
  geom_line(color = "firebrick") +
  labs(title = "Trend (Tt)", y = "Trend", x = "") +
  theme_minimal()

p_seasonal <- permit_final %>%
  ggplot(aes(x = Month, y = Seasonal)) +
  geom_line(color = "darkgreen") +
  labs(title = "Seasonal (St)", y = "Index", x = "") +
  theme_minimal()

p_remainder <- permit_final %>%
  ggplot(aes(x = Month, y = Remainder)) +
  geom_line(color = "tomato") +
  geom_hline(yintercept = 1, linetype = "dashed") +
  labs(title = "Remainder (Rt)", y = "Ratio", x = "Year") +
  theme_minimal()
```

Ordering the variables: Data » Trend » Seasonal » Remainder

Storing as plot1 using patchwork for a vertical stack or grid

```
plot1 <- (p_data / p_trend / p_seasonal / p_remainder) +  
  plot_annotation(title = "Multiplicative Decomposition: VABPPRIV")
```

- (i) make use of the `pivot_longer()` function to reshape your data. You can select the 4 columns of interest here.
- (ii) convert your variable names to **factors** so that R respects the “hierarchy” (that I requested) and is not ordered alphabetically in the faceted graphs.
- (iii) use the `facet_wrap()` function to stack your plots in a single column (so they resemble the results of the built-in function).
- (iv) use the `ggtitle()` function to add the title “My Decomposition” to your plot.

```
plot1 <- permit_final %>%  
  as_tibble() %>%  
  select(Month, VABPPRIV, Trend_Cycle, Seasonal, Remainder) %>%  
  pivot_longer(-Month, names_to = "component", values_to = "value") %>%  
  mutate(component = factor(component,  
    levels = c("VABPPRIV", "Trend_Cycle", "Seasonal", "Remainder"),  
    labels = c("Data", "Trend", "Seasonal", "Remainder"))) %>%  
  ggplot(aes(x = Month, y = value)) +  
  geom_line(color = "steelblue") +  
  facet_wrap(~component, ncol = 1, scales = "free_y") +  
  ggtitle("My Decomposition") +  
  labs(x = "Year", y = NULL) +  
  theme_minimal()
```

Step v(b). Use the `classical_decomposition()` function, within the `model()` function, to create R’s version of the **multiplicative** decomposition of the series. Store this plot as `plot2`. Again, do not present the plot here.

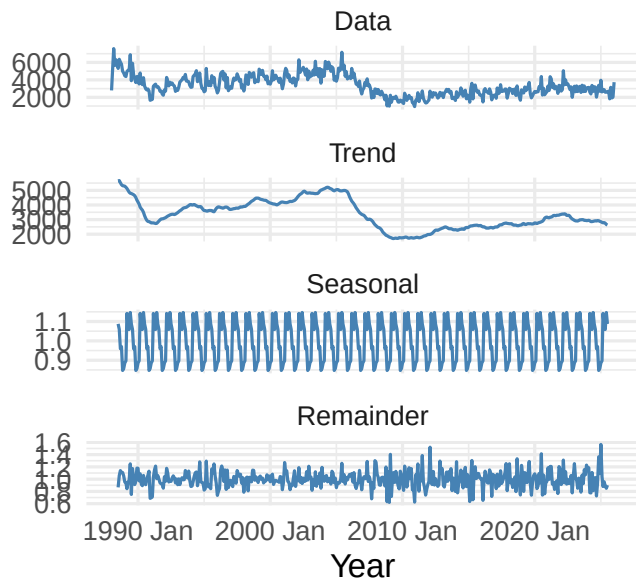
- Add the title “R’s Decomposition” to your plot.

```
fit_classical <- permit %>%  
  model(classical = classical_decomposition(VABPPRIV, type = "multiplicative"))  
  
plot2 <- fit_classical %>%  
  components() %>%  
  autoplot() +  
  # Add the requested title  
  ggtitle("R's Decomposition") +  
  theme_minimal()
```

Step vi. Present a 1x2 plot of `plot1` and `plot2`.

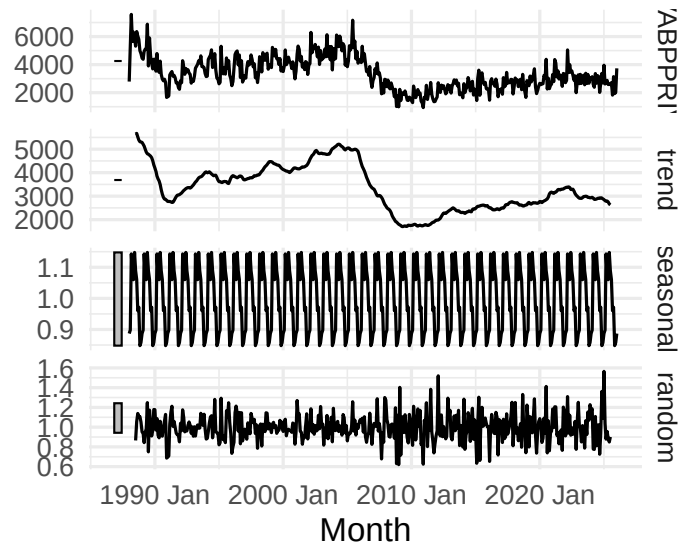
```
plot1 + plot2
```


My Decomposition



R's Decomposition

VABPPRIV = trend * seasonal * random



4. Now, confirm that your decomposition was correct by **plotting the multiplication of the three extracted components (on y-axis) against the actual data (on x-axis)**.

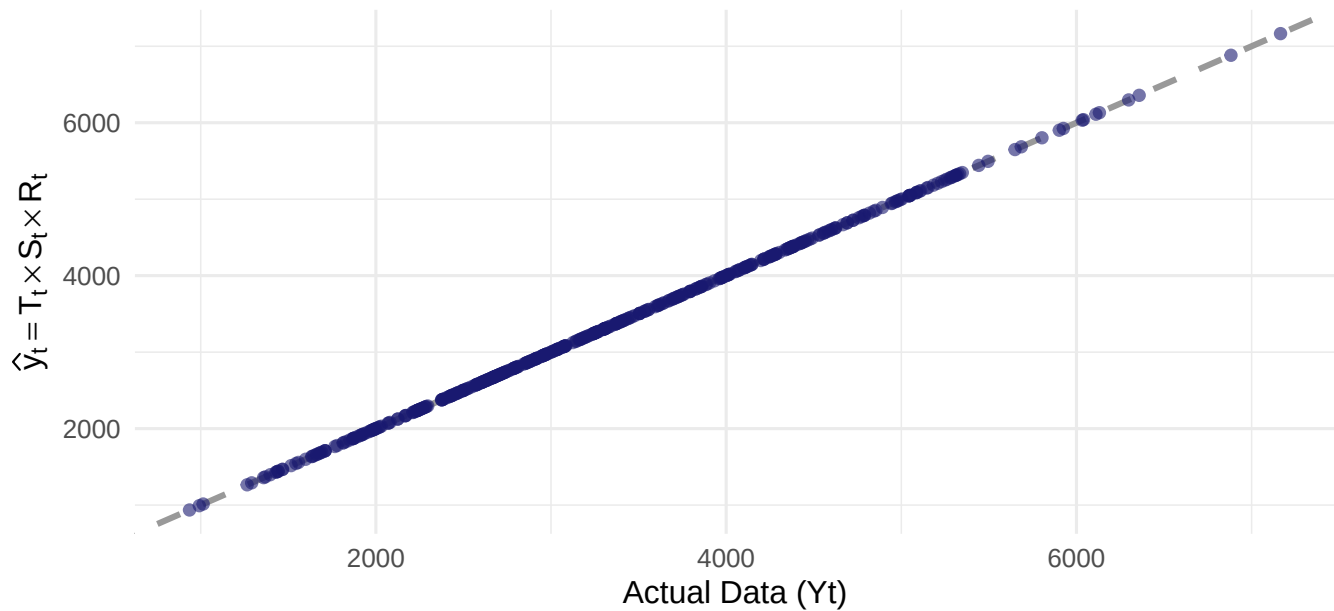
- Be sure to add in a 45° line to see how the scatter plot stacks up against this line.
- Produce this plot using the `ggplot` function.
- Get fancy with it. Add a y-axis title that uses the `bquote()` function to display the $\hat{y}$ symbol.

```
permit_check <- permit_final %>%
  mutate(y_hat = Trend_Cycle * Seasonal * Remainder) %>%
  filter(!is.na(y_hat))

ggplot(permit_check, aes(x = VABPPRIV, y = y_hat)) +
  geom_abline(slope = 1, intercept = 0, color = "gray60", linetype = "dashed", size = 1) +
  geom_point(alpha = 0.6, color = "midnightblue") +
  labs(title = "Validation of Classical Multiplicative Decomposition",
        subtitle = "Comparing Predicted Product to Actual Observations",
        x = "Actual Data (Yt)",
        y = bquote(widehat(y)[t] == T[t] %*% S[t] %*% R[t])) +
  theme_minimal()
```

Validation of Classical Multiplicative Decomposition

Comparing Predicted Product to Actual Observations

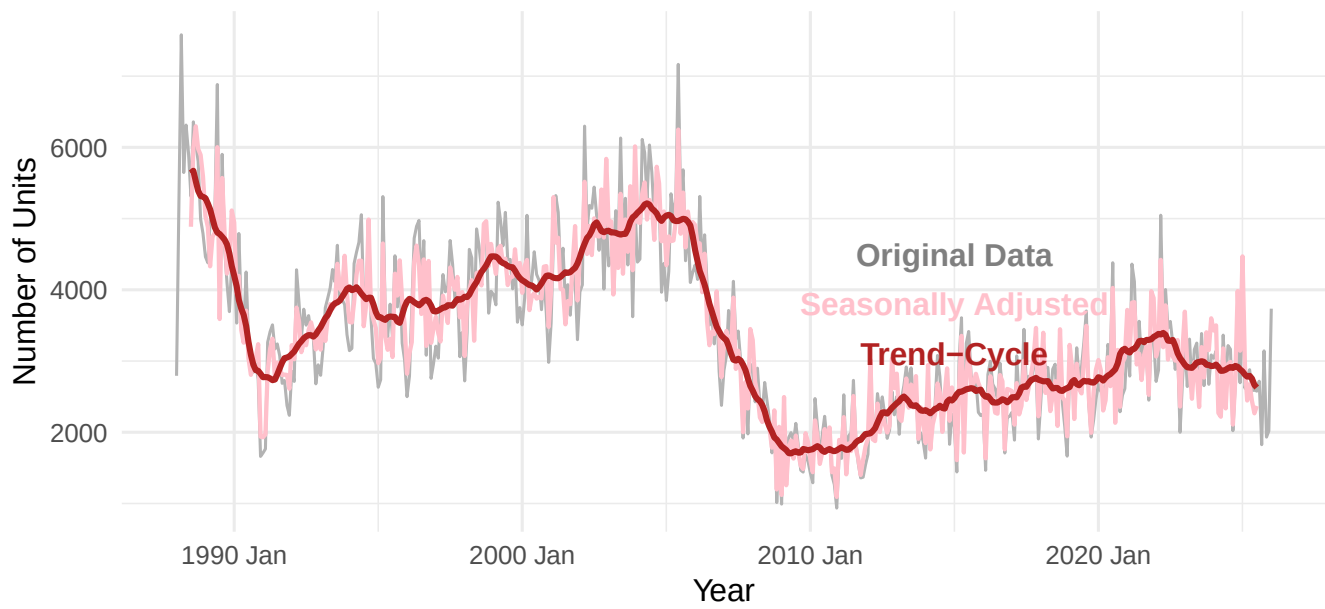


5. Produce an `autoplot()` of the actual data, permit, and `autolayer()` your computed `trend-cycle` and `seasonally-adjusted` variables.

Ensure that your series are correctly labeled. You might also need to change the colors of each of the series.

```
permit_final %>%
  autoplot(VABPPRIV, color = "gray70") +
  autolayer(permit_final, Seasonally_Adjusted, color = "pink", size = 0.7) +
  autolayer(permit_final, Trend_Cycle, color = "firebrick", size = 1) +
  labs(title = "Original, Seasonally Adjusted, and Trend-Cycle Data",
        subtitle = "Virginia Housing Permits",
        y = "Number of Units",
        x = "Year") +
  annotate("text", x = yearmonth("2015 Jan"), y = 4500, label = "Original Data", color = "gray50", fontface = "bold", size = 10) +
  annotate("text", x = yearmonth("2015 Jan"), y = 3800, label = "Seasonally Adjusted", color = "pink", fontface = "bold", size = 10) +
  annotate("text", x = yearmonth("2015 Jan"), y = 3100, label = "Trend-Cycle", color = "firebrick", fontface = "bold", size = 10) +
  theme_minimal()
```

Original, Seasonally Adjusted, and Trend–Cycle Data Virginia Housing Permits



Q2: Forecasting Daily Minimum Temperatures with STL Decomposition

In this exercise, we will work with daily minimum temperatures recorded in Melbourne, Australia, spanning January 1981 through December 1990.

1. Load the dataset `daily-min-temperatures.csv` from our Data folder on GitHub and store it as a `tsibble` object called `temp`. The `Date` column should be parsed using `ymd()` and used as the index.

```
data_url <- "https://raw.githubusercontent.com/Shamar-Stewart/Forecasting-S26/refs/heads/main/Data/daily-min-temperatures.csv"

temp <- read_csv(data_url) %>%
  mutate(Date = ymd(Date)) %>%
  as_tsibble(index = Date)

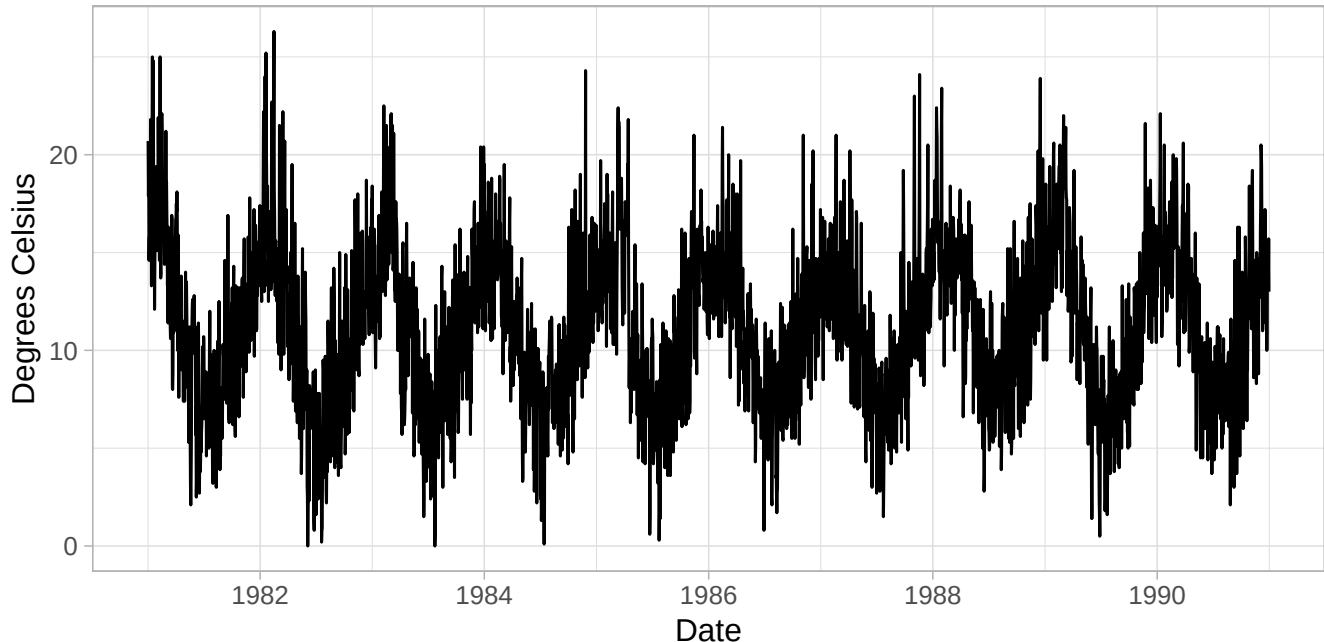
temp
```

```
## # A tsibble: 3,650 x 2 [1D]
##   Date      Temp
##   <date>    <dbl>
## 1 1981-01-01 20.7
## 2 1981-01-02 17.9
## 3 1981-01-03 18.8
## 4 1981-01-04 14.6
## 5 1981-01-05 15.8
## 6 1981-01-06 15.8
## 7 1981-01-07 15.8
## 8 1981-01-08 17.4
## 9 1981-01-09 21.8
## 10 1981-01-10 20
## # i 3,640 more rows
```

2. Produce an `autoplot()` of the `temp` series. Briefly comment on any patterns you observe. Does the series appear to have a trend? What can you say about the seasonality?

```
temp %>%
  autoplot(Temp) +
  labs(title = "Daily Minimum Temperatures in Melbourne",
       y = "Degrees Celsius", x = "Date")
```

Daily Minimum Temperatures in Melbourne



I can see that on this plot that this is cyclical making it stationary. There is a pattern of upward and decreasing cycles but it seems you would not be able to be too specific. There seems to be a slight decreasing trend with more tight variance in the second half of the data. You can see seasonality with rises along summer months and dips in winter months *in southern hemisphere months*.

3. Now **attempt** to produce a `gg_season()` plot of `temp`. **Note: You should encounter a `fill_gaps()` error.**

```
#temp %>%  
  #gg_season(Temp)
```

Read the error message carefully. It is telling you that your data contains **implicit gaps** in the time index — meaning at least one date is absent from the dataset (there is no row for it whatsoever). This is different from having a row with a missing (NA) value. The error is also helpfully suggesting a potential fix: `fill_gaps()`.

- (a) To confirm how many gaps exist and where use the `has_gaps()` and `count_gaps()` functions. Report your findings. How many gaps are there, and on which dates do they fall?

There are two explicit gaps in the data. One is at the date 1984-12-31 and another at 1988-12-31, respectively.

```
has_gaps(temp)
```

```
## # A tibble: 1 x 1  
##   .gaps  
##   <lgl>  
## 1 TRUE
```

```
count_gaps(temp)
```

```
## # A tibble: 2 x 3  
##   .from      .to      .n  
##   <date>    <date>    <int>  
## 1 1984-12-31 1984-12-31      1  
## 2 1988-12-31 1988-12-31      1
```

- (b) Now fix the issue using the following code, which first inserts the missing dates into the index and then fills the resulting NA values using the **last observed value** (carry-forward):

```
temp <- temp %>%
  fill_gaps() %>%
  fill(Temp, .direction = "down")

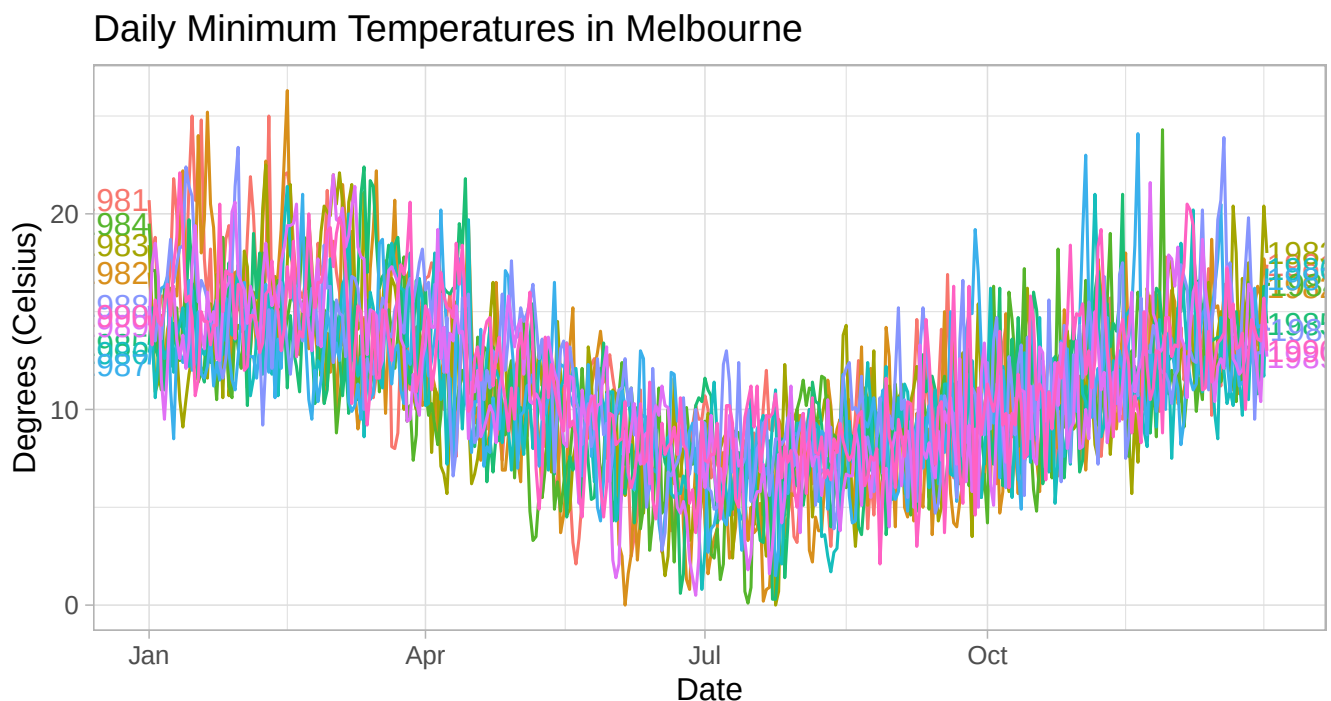
temp %>% has_gaps()
```

```
## # A tibble: 1 x 1
##   .gaps
##   <lgl>
## 1 FALSE
```

Now reproduce the `gg_season()` plot and comment on the seasonal pattern you observe. What does the shape of the seasonal cycle tell you about Melbourne's climate?

I see a seasonal decrease in daily minimum temperatures specifically when observing our summer months which is their winter months. This is just showing how the southern hemisphere has flipped seasonal trends compared to the northern hemisphere.

```
temp %>%
  gg_season(Temp, labels = "both") +
  labs(title = "Daily Minimum Temperatures in Melbourne",
       y = "Degrees (Celsius)")
```



4. Your classmate, William, argues that an **additive** decomposition is appropriate for this series. Do you agree with his assessment? Be sure to explain your reasoning clearly — a simple yes or no will not suffice. (*Hint: think carefully about how the magnitude of the seasonal fluctuations behaves relative to the level of the series.*)

I agree with William because of the seemingly stable seasonal variance over the course of the plot regardless of the way the plot is going indicating an additive decomposition should be used.

5. Your colleague is convinced that the X-11 and SEATS would be appropriate decomposition methods for this dataset. You strongly disagree. Briefly explain why STL is the more suitable choice here, drawing on what you know about the limitations of X-11 and SEATS.

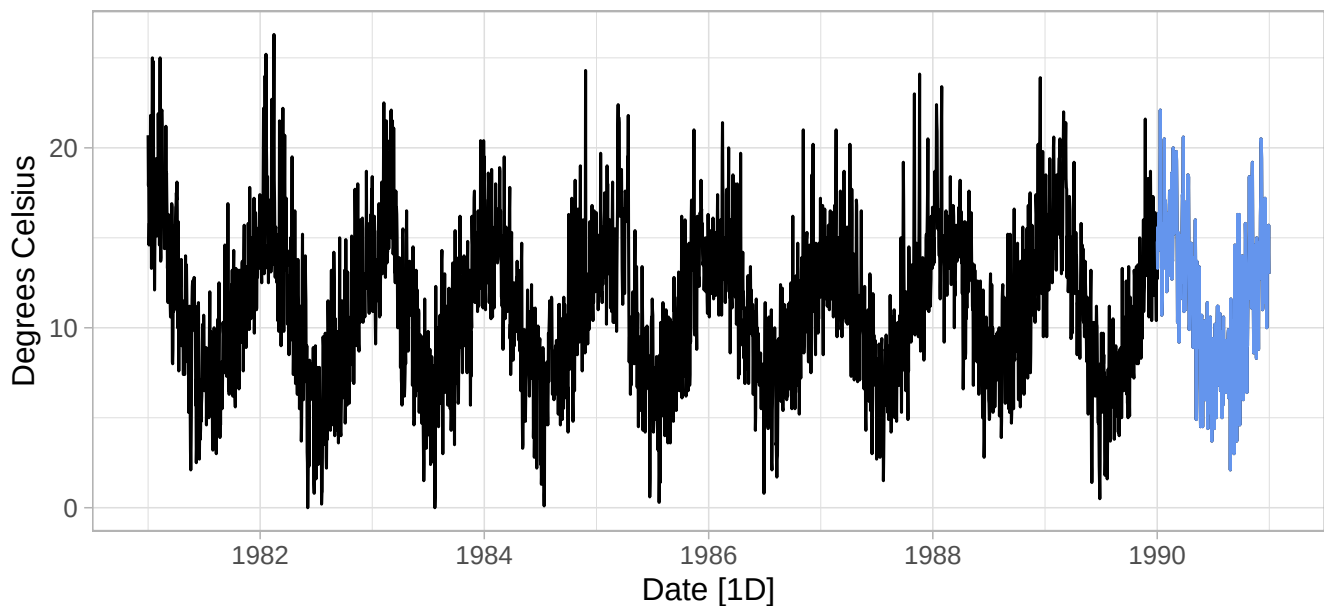
X-11 and SEATS are more for monthly and quarterly split data. The STL has more robustness to outliers, which is good for sets like temperature because climate events can swing the data to one side if bad enough and STL is more fit to handle these.

6. Now assign the values in `temp` up to and including December 31, 1989 as a training set called `temp.train`. Assign the remaining values to `temp.test`. Produce a plot clearly showing the train/test split.

```
temp.train <- temp %>%  
  filter(year(Date) <= 1989)  
  
temp.test <- temp %>%  
  filter(year(Date) > 1989)  
  
temp %>%  
  autoplot(Temp) +  
  autolayer(temp.test, Temp, color = "cornflowerblue") +  
  labs(title = "Melbourne Daily Min Temperatures",  
        subtitle = "Black: Training (1981-1989), Red: Test (1990)",  
        y = "Degrees Celsius")
```

Melbourne Daily Min Temperatures

Black: Training (1981-1989), Red: Test (1990)



7. Using `temp.train`, fit an STL decomposition with **fixed seasonal windows** and `robust = TRUE`. Store the result as `fit1` and present the decomposition plot with an appropriate title.

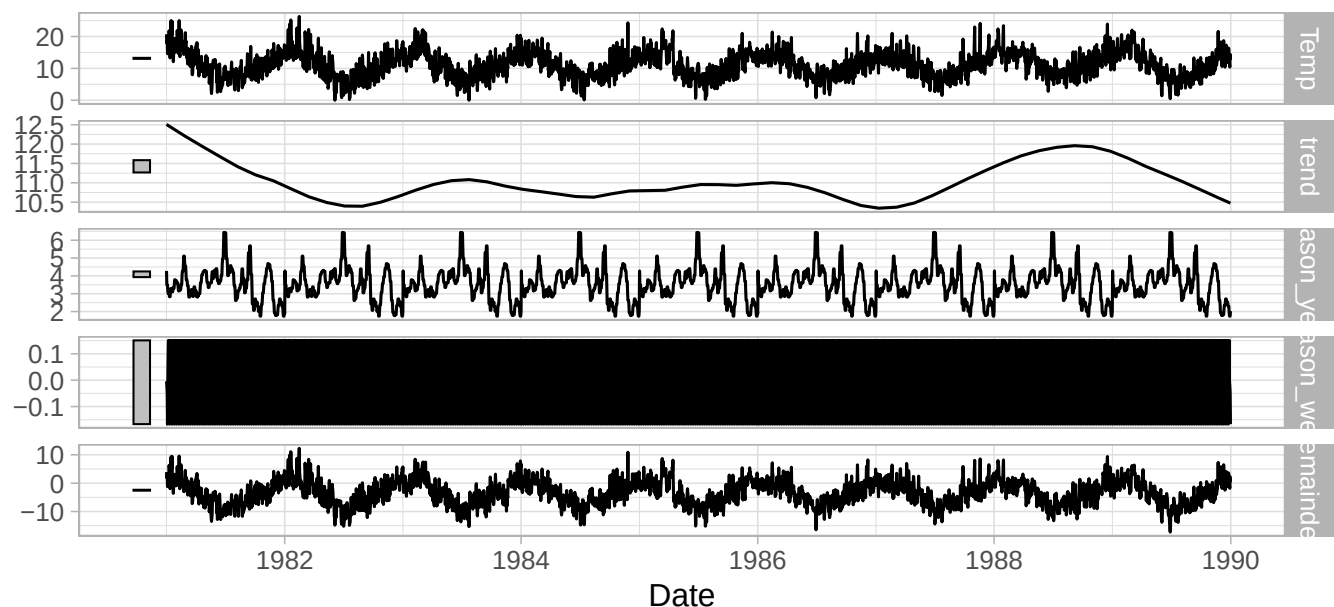
The STL function is able to handle complex (multiple layers of) seasonality. You will notice that the `components` function produces two seasonal components: `season_year` and `season_week`. The former captures seasonality that repeats every year, while the latter captures seasonality that repeats every week. The `season_year` component is likely capturing the annual temperature cycle, while the `season_week` component may be capturing any weekly patterns in the data (e.g., differences between weekdays and weekends).

```
fit1 <- temp.train %>%
  model(stc = STL(Temp ~ season(period = "year", window = "periodic") +
    season(period = "week", window = "periodic"),
    robust = TRUE))

fit1 %>%
  components() %>% autoplot() +
  labs(title = "STL Decomposition of Melbourne Daily Min Temperatures",
    subtitle = "Fixed seasonal windows (window = 'periodic')")
```

STL Decomposition of Melbourne Daily Min Temperatures

Fixed seasonal windows (window = 'periodic')



8. Comment on the decomposition. How would you describe the trend-cycle component? Do you observe any seasonality? Do you notice anything peculiar about the remainder component? What could it be signalling here?

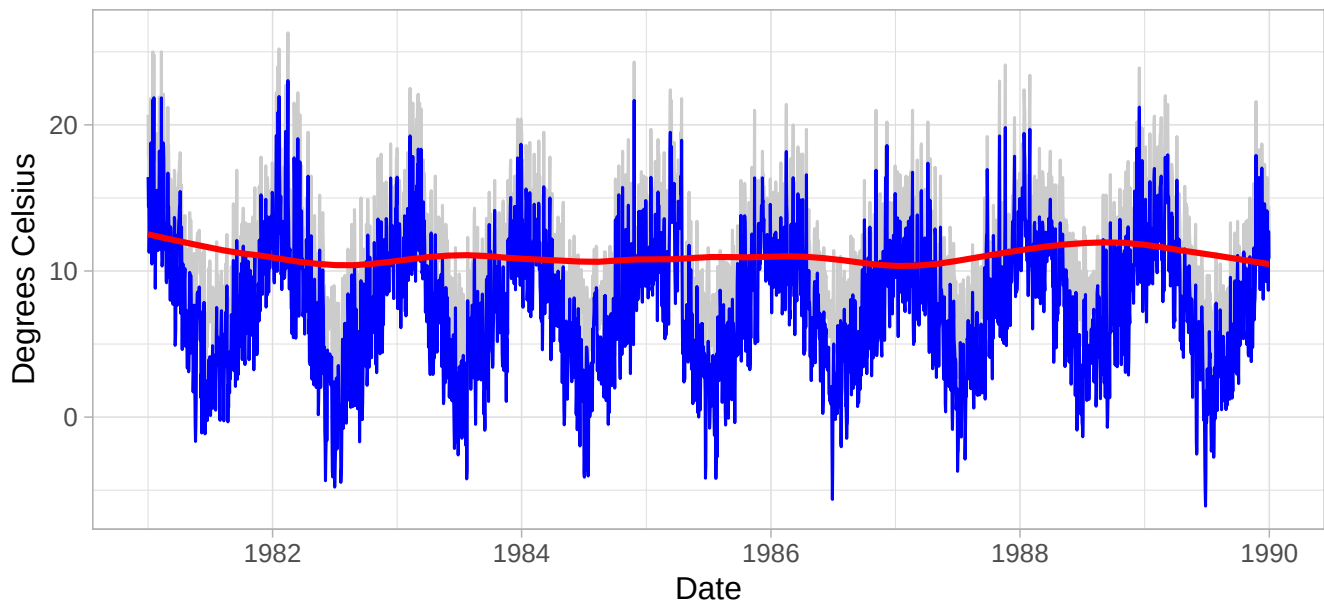
The trend-cycle component has some general movements with a larger peak later in the plot but has no real specified direction. The seasonal yearly parameter shows clear seasonality with peaks and troughs in their respective summer and winter months. The remainder components still seems to have noticeable spikes at certain times and could represent the result of “out of season” events like droughts, for example, being pushed into this parameter.

9. In a single graph, superimpose the **seasonally adjusted** and **trend-cycle** components onto the training data. Be sure to use distinct (and visible) colors and include an appropriate title.

```
fit1 %>%
  components() %>%
  mutate(season_adj = Temp - season_year - season_week) %>%
  ggplot(aes(x = Date)) +
  geom_line(aes(y = Temp), color = "gray80") +
  geom_line(aes(y = season_adj), color = "blue") +
  geom_line(aes(y = trend), color = "red", size = 1) +
  labs(title = "Melbourne Temperatures: Seasonally Adjusted and Trend",
    subtitle = "Original (Gray), Seasonally Adjusted (Blue), and Trend (Red)",
    y = "Degrees Celsius")
```


Melbourne Temperatures: Seasonally Adjusted and Trend

Original (Gray), Seasonally Adjusted (Blue), and Trend (Red)

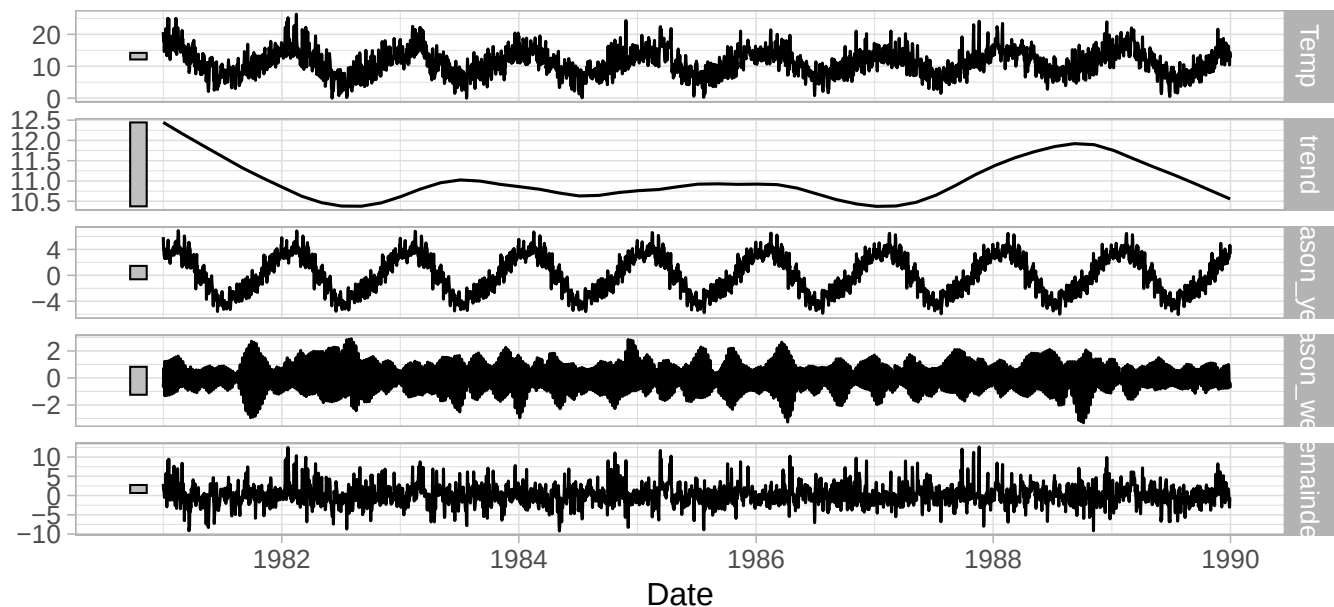


10. Now fit a second STL model on `temp.train`, this time allowing the seasonal and trend windows to be selected automatically by R, with `robust = TRUE`. Store this as `fit2` and present its decomposition plot.

```
fit2 <- temp.train %>%  
  model(stc = STL(Temp ~ season(period = "year") +  
    season(period = "week"),  
    robust = TRUE))  
  
fit2 %>%  
  components() %>%  
  autoplot() +  
  labs(title = "STL Decomp | AWS",  
    subtitle = "robust = TRUE")
```

STL Decomp | AWS

robust = TRUE



Present the two decompositions (fit1 and fit2) as a side-by-side plot using patchwork. Give each plot an appropriate title and comment on any discernible differences between the two, including any differences in the magnitude of the seasonal component.

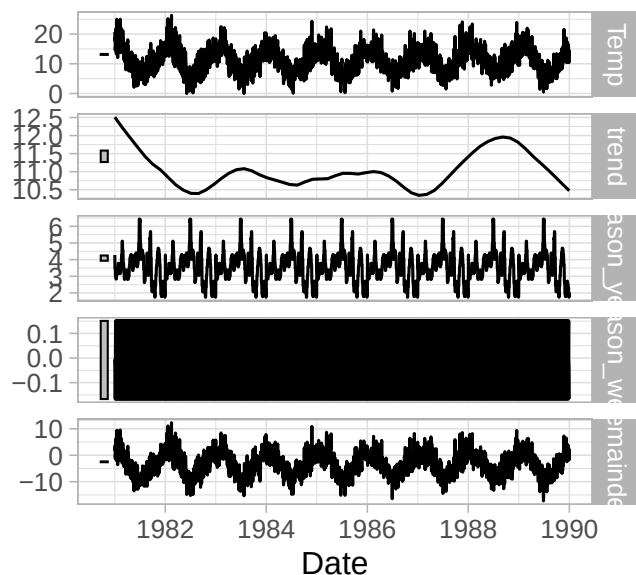
```
p1 <- fit1 %>% components() %>% autoplot() +  
  labs(title = "Fit 1: Fixed (Periodic) Windows")
```

```
p2 <- fit2 %>% components() %>% autoplot() +  
  labs(title = "Fit 2: Automatic Windows")
```

```
p1 | p2
```

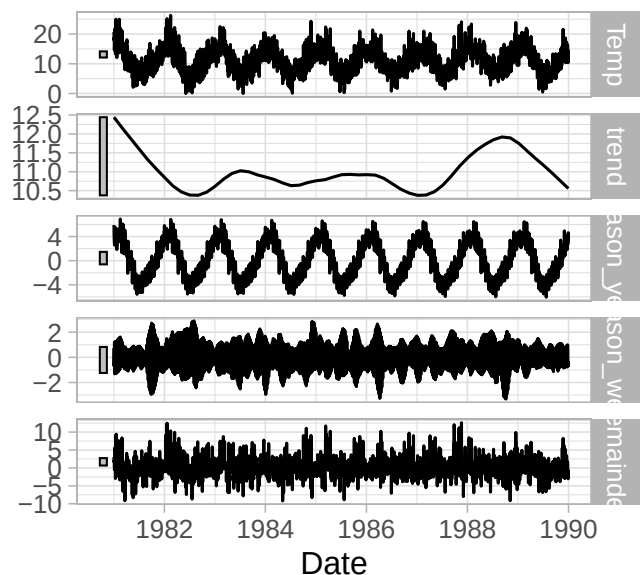
Fit 1: Fixed (Periodic) Windows

Temp = trend + season_year + season_week + remainder



Fit 2: Automatic Windows

Temp = trend + season_year + season_week + remainder



There seems to be thicker/ more variance in the remainder component in the fixed plot. The temperature slots on

both seem to be very similar. A key difference is the weekly seasonal areas where the automatic plot shows more of a zoomed out view with more information to be seen as to the variance.

11. Using the **fixed seasonality** model in Part 7,

- i. use an random walk with drift model to produce predictions of the **seasonally adjusted** portion of `temp` over the length of the test period. Store your results in a variable called `arima.temp`.

```
seasonally_adj_data <- fit1 %>%
  components() %>%
  mutate(season_adj = Temp - season_year - season_week) %>%
  select(Date, season_adj)

arima.temp <- seasonally_adj_data %>%
  model(drift_mod = RW(season_adj ~ drift())) %>%
  forecast(h = 365)
```

- ii. produce a forecast of each seasonal component (`season_year` and `season_week`) over the test period using the **seasonal naïve** method. Save these as `snaive.seas.year` and `snaive.seas.week` respectively.

```
comp_clean <- fit1 %>%
  components() %>%
  as_tsibble() %>%
  select(Date, season_year, season_week)

snaive.seas.year <- comp_clean %>%
  model(s_year = SNAIVE(season_year)) %>%
  forecast(h = 365)

snaive.seas.week <- comp_clean %>%
  model(s_week = SNAIVE(season_week)) %>%
  forecast(h = 365)
```

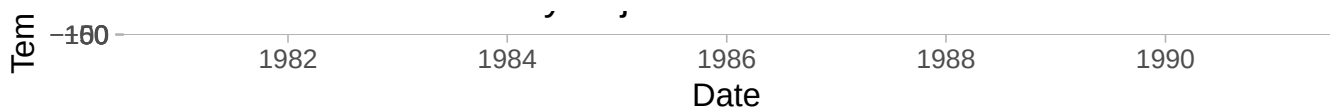
- iii. produce a gridded plot (as a 3X1) of the forecasted of both methods over the test period.

```
p_drift <- arima.temp %>%
  autoplot(seasonally_adj_data) +
  labs(title = "Drift Forecast: Seasonally Adjusted", y = "Temp")

p_year <- snaive.seas.year %>%
  autoplot(comp_clean) +
  labs(title = "SNAIVE Forecast: Annual Seasonality", y = "Temp")

p_week <- snaive.seas.week %>%
  autoplot(comp_clean) +
  labs(title = "SNAIVE Forecast: Weekly Seasonality", y = "Temp")

p_drift / p_year / p_week
```



level 80% 95%



level 80% 95%



level 80% 95%

12. Combine the forecasts from Part 11 to produce a reseasonalized forecast of the original temperature data. Store this as `pred.temp` and produce a plot displaying the reseasonalized forecast alongside the actual test data (`temp.test`).

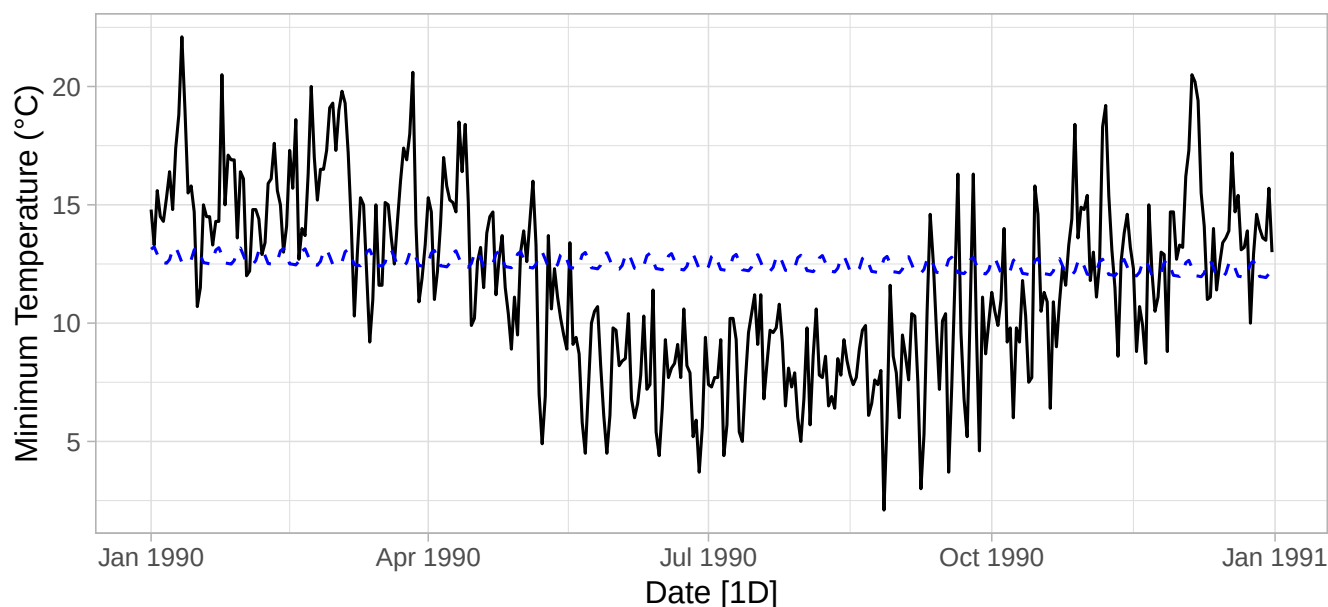
Hint: You might find it helpful to (i) convert `drift.temp`, `snaive.seas.year`, and `snaive.seas.week` to tibble objects; (ii) select the `Date` and `.mean` columns from each; (iii) join all three tibbles by the `Date` column; (iv) sum the three `.mean` columns to get the reseasonalized forecast.

```
pred.temp <- arima.temp %>%
  as_tibble() %>%
  select(Date, drift_mean = .mean) %>%
  left_join(as_tibble(snaive.seas.year) %>% select(Date, year_mean = .mean),
    by = "Date") %>%
  left_join(as_tibble(snaive.seas.week) %>% select(Date, week_mean = .mean),
    by = "Date") %>%
  mutate(Total_Pred = drift_mean + year_mean + week_mean) %>%
  as_tsibble(index = Date)

temp.test %>%
  autoplot(Temp) +
  autolayer(pred.temp, Total_Pred, color = "blue", linetype = "dashed") +
  labs(title = "Reseasonalized STL Forecast vs. Actual Data (1990)",
    subtitle = "Blue dashed line as Combined STL Forecast | Black line as Actual 1990 Data",
    y = "Minimum Temperature (°C)")
```

Reseasonalized STL Forecast vs. Actual Data (1990)

Blue dashed line as Combined STL Forecast | Black line as Actual 1990 Data



13. Briefly discuss your observations from the plot above. How well does this forecast capture the dynamics of the actual test data? Any insight into how the model could be improved?

It does a great job of capturing the seasonal elements and has the same “u-shaped” plot of the actual test data. There are high variances still could maybe be paired with additional models to smooth out the volatile lines.

14. Using the `decomposition_model()` function and both the fixed and changing seasonality STL models (with `robust = TRUE`), fit the following six models on `temp.train`. Store all models in a single `model()` call as `mod.fit`, then store the forecasts over the test period as `mod.all`. The six models are:

- RW with drift (seasonally adjusted) + Seasonal Naïve (both seasonal components), fixed seasonality
- RW with drift (seasonally adjusted) + Seasonal Naïve (both seasonal components), changing seasonality
- RW with drift (seasonally adjusted) + ARIMA (both seasonal components), fixed seasonality
- RW with drift (seasonally adjusted) + ARIMA (both seasonal components), changing seasonality
- ARIMA (seasonally adjusted) + ARIMA (both seasonal components), fixed seasonality
- ARIMA (seasonally adjusted) + ARIMA (both seasonal components), changing seasonality

Note: Please name all six models appropriately. You are not required to plot the forecasts here.

```
stl_fixed <- STL(Temp ~ season(period = "year", window = "periodic") +
  season(period = "week", window = "periodic"), robust = TRUE)
```

```
stl_changing <- STL(Temp ~ season(period = "year") +
  season(period = "week"), robust = TRUE)
```

```
mod.fit <- temp.train %>%
```

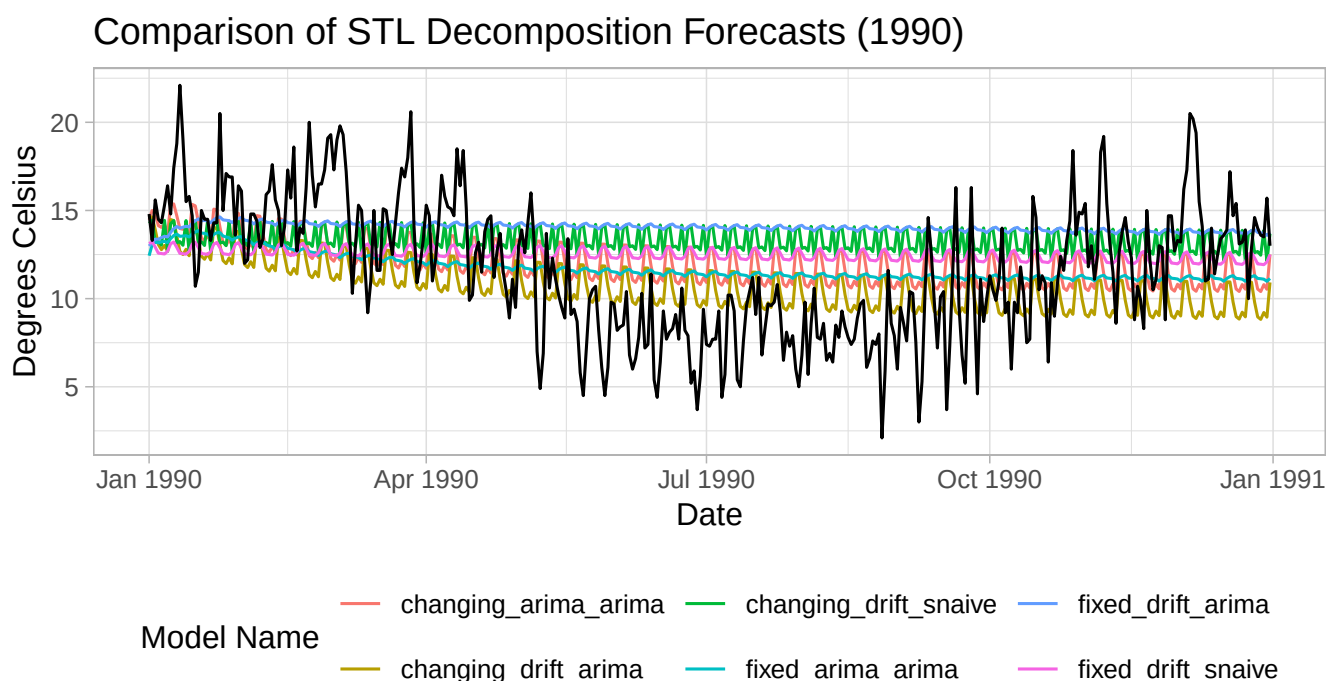
```
  model(fixed_drift_snaive = decomposition_model(stl_fixed, RW(season_adjust ~ drift()), SNAIVE(season_year),
  changing_drift_snaive = decomposition_model(stl_changing, RW(season_adjust ~ drift()), SNAIVE(season_year),
  fixed_drift_arima = decomposition_model(stl_fixed, RW(season_adjust ~ drift()), ARIMA(season_year), ARIMA(season_year),
  changing_drift_arima = decomposition_model(stl_changing, RW(season_adjust ~ drift()), ARIMA(season_year),
  fixed_arima_arima = decomposition_model(stl_fixed, ARIMA(season_adjust), ARIMA(season_year), ARIMA(season_year))
```

```
changing_arima_arima = decomposition_model(stl_changing, ARIMA(season_adjust), ARIMA(season_year), ARIMA(s
mod.all <- mod.fit %>%
  forecast(h = 365)
```

- Produce a plot of all six model forecasts alongside the **test data**. Turn the prediction intervals off. Which model appears to do the best job visually?

I cannot decide which model really is the best. They seem to all have a similar problem of being too smooth. The changing arima stays around the mean the best so I will go that the best model is the changing ARIMA.

```
mod.all %>%
  autoplot(temp.test, level = NULL) +
  labs(title = "Comparison of STL Decomposition Forecasts (1990)",
       y = "Degrees Celsius",
       color = "Model Name") +
  theme(legend.position = "bottom")
```



- Compute forecast accuracy for all six models. Present your results as a **kable** table rounded to 3 decimal places, reporting RMSE and MAPE. Which model is preferred by each metric? Be sure to explain how you arrived at your conclusion.

The RMSE parameter would be best with the fixed arima model and the MAPE would select the changing drift model. We are looking for the lowest RMSE and lowest MAPE values respectively.

```
mod.all %>%
  accuracy(temp.test) %>%
  select(.model, RMSE, MAPE) %>%
  arrange(RMSE) %>%
  kable(digits = 3, caption = "Forecast Accuracy for STL Decomposition Models")
```

Table 1: Forecast Accuracy for STL Decomposition Models

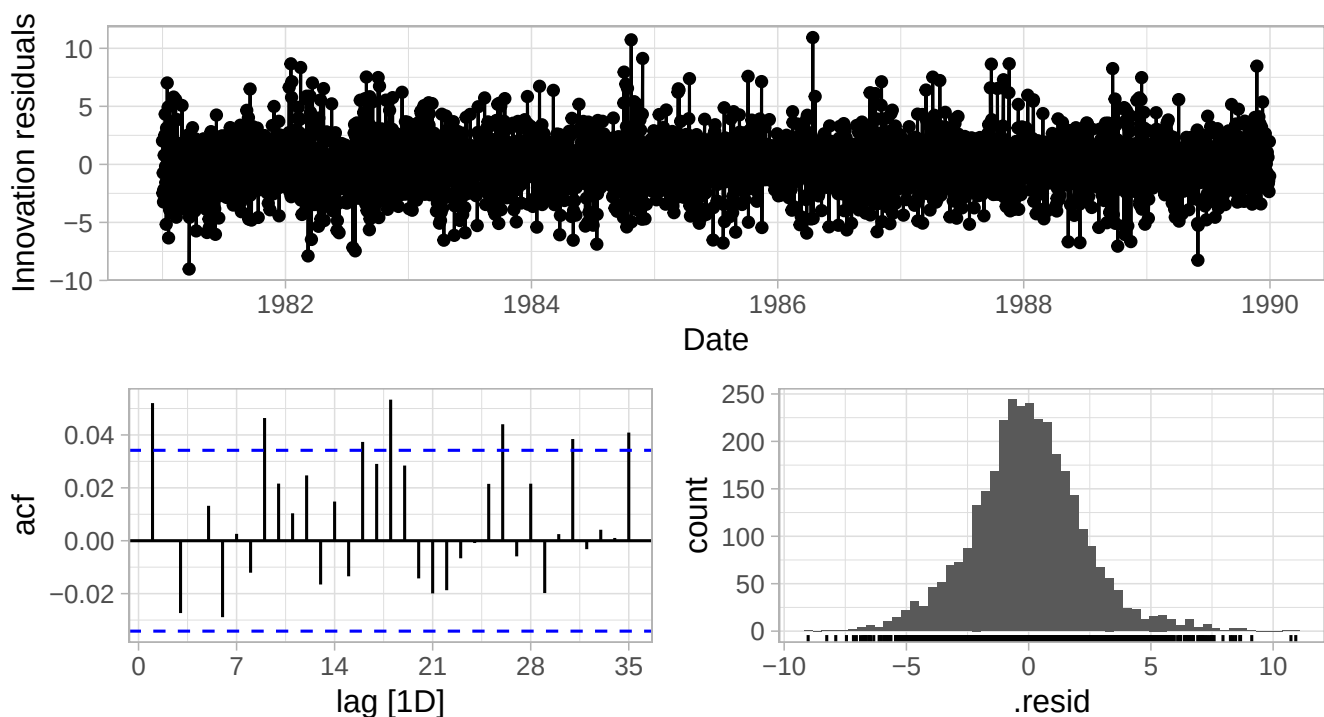
.model	RMSE	MAPE
fixed_arma_arma	3.55	32.0
changing_arma_arma	3.67	33.5
changing_drift_arma	3.77	30.2
fixed_drift_snaive	3.92	37.6
changing_drift_snaive	4.25	41.9
fixed_drift_arma	4.51	45.5

17. Using your preferred model from above, produce a residual diagnostic plot using `gg_tsresiduals()` and conduct a Ljung-Box test with a lag of 7. Do the residuals appear to be white noise? Discuss your findings with reference to both the plot and the test result.

The residuals do not appear to be white noise as there are spikes at certain lags that cross the band consistently leaving information on the table that we could take advantage of. The distribution chart seems to be normally distributed with a good bell curve and potentially some outliers on both sides of the tails. The p-value of .0397 compared to the significance value of .05 and the p-value being lower means we have enough evidence to reject the null hypothesis of it being white noise and say that it is not white noise.

```
best_fit <- mod.fit %>% select(changing_arma_arma)
```

```
best_fit %>% gg_tsresiduals()
```



```
best_fit %>%
  augment() %>%
  features(.innov, ljung_box, lag = 7)
```

```
## # A tibble: 1 x 3
##   .model          lb_stat lb_pvalue
##   <chr>          <dbl>   <dbl>
## 1 changing_arma_arma    14.7    0.0397
```